

基于进迭时空 K1 的”小蝶”端侧儿童语音大模型系统

摘要——”小蝶”是一套部署在进迭时空 K1 MUSE Pi Pro 板卡上的端侧儿童语音故事系统。针对现有儿童故事机内容固化、云端语音助手隐私与网络依赖等不足，本作品在 Bianbu Linux 平台上完成了从语音输入到语音输出的全链路端侧 AI 部署。系统以 GPIO 按钮触发录音，经 USB 麦克风采集 16kHz 单声道音频，由本地常驻的 sherpa-onnx Zipformer int8 ASR 守护进程完成离线语音识别；识别文本经 BM25 轻量检索引擎匹配动画 IP 知识卡片，注入安全约束与角色设定后，送入基于 Qwen2.5-1.5B-Instruct 微调并量化为 Q4_K_M GGUF 格式的本地大语言模型，由 Ollama 框架驱动流式生成儿童故事文本；生成的文本按完整中文句分段，逐段送入本地 Chaowen Full TTS C 守护进程合成语音，经 ffmpeg 转码后通过 3.5mm 耳机口播放。Qt6 原生界面同步显示儿童可理解的交互状态与故事内容，支持按钮打断与重新发起。

训练阶段，项目收集 TinyStories Chinese 等开放许可数据集共 5455 条样本，经 HTML 清洗、安全内容过滤与近重复去重后构建 5180 条 SFT 样本，在 RTX 4060 Ti 上以 LoRA ($r = 16, \alpha = 32$) 方式微调 2000 步，最佳 eval loss 为 1.882。模型合并为 FP16 Hugging Face 格式后，经 llama.cpp 转换并量化为 Q4_K_M GGUF，最终模型体积约 986MB。端侧部署采用 ASR/TTS 常驻守护进程与模型预热策略：纯 LLM 推理首 token 延迟约 2.9s，生成速度约 3.3tokens/s；RAG 缓存命中后首 token 延迟约 4.8s；LLM 与 TTS 并发运行时生成速度约 1.6tokens/s，通过句子级流式 TTS 覆盖等待。系统成本低、功耗有限、数据不出本地，适合智能故事机、儿童陪伴机器人和幼教互动终端等场景。

I. 作品概述

A. 功能与特性

”小蝶”系统提供完整的端侧语音交互：儿童长按实体按钮说出故事主题，松开后系统依次完成语音识别、知识增强、故事生成与语音合成，最终通过耳机播放个性化的儿童故事。

系统的主要功能包括：（1）端侧离线语音识别，基于 sherpa-onnx Zipformer int8 模型，将儿童语音转为文本；（2）动画 IP 知识增强，通过 BM25 检索 156 张 IP 知识卡片（涵盖小猪佩奇、汪汪队、海底小纵队等），在故事生成时注入角色设定与安全约束；（3）端侧

大语言模型故事生成，使用 Qwen2.5-1.5B-Instruct 经 LoRA 微调并量化为 Q4_K_M GGUF 后在 K1 上运行；（4）端侧中文语音合成，采用 Chaowen Full TTS 实现离线高质量语音输出；（5）Qt6 原生界面，提供”正在听””正在想故事””讲故事中”等儿童友好的状态显示；（6）按钮打断机制，在任何阶段均可按下按钮重新开始新的故事。

上述链路无需互联网连接，所有 AI 推理均在 K1 板卡本地完成。

B. 应用领域

本系统面向 3-7 岁学龄前儿童的语音陪伴与启蒙阅读场景。

智能故事机。传统故事机预置固定音频内容，儿童无法根据自身兴趣定制故事。”小蝶”系统通过端侧大模型实时生成故事，儿童只需说出”佩奇和乔治整理玩具”等简短主题，即可获得对应的个性化故事，并以自然语音播放。

儿童陪伴机器人。系统可嵌入到具有外壳和扬声器的硬件产品中，作为儿童陪伴机器人的语音交互核心。端侧部署特性使其无需依赖云端服务器，适合家庭和幼教机构使用。

幼教互动终端。RAG 知识库可扩展为科普、安全教育、生活习惯培养等不同主题的卡片集合，由此将系统的应用范围从故事娱乐拓展到教育引导。

RISC-V 端侧 AI 应用示范。本项目验证了在 K1 RISC-V CPU 上运行 1.5B 参数大语言模型、离线 ASR 和离线 TTS 的可行性，为同类端侧 AI 产品提供了工程参考。

C. 主要技术特点

全链路端侧 AI 推理。ASR、LLM 与 TTS 三个 AI 模块均在 K1 板卡本地离线运行，不依赖云端 API。ASR 使用 sherpa-onnx C API 封装的 Zipformer int8

模型，LLM 使用 Ollama 驱动的 Q4_K_M 量化 GGUF 模型，TTS 使用 sherpa-onnx C API 封装的 Chaowen Full VITS 模型。

轻量 RAG 知识增强。采用 BM25 文本检索替代向量 embedding 模型，在 K1 端无需额外的 embedding 推理开销。知识卡片包含角色关系、别名、安全策略等结构化信息，检索时按 franchise 过滤并强制注入安全类卡片。

常驻守护进程架构。ASR 与 TTS 均以 C 语言编写的 daemon 方式运行，程序启动时一次性加载模型，后续交互无需重复加载。ASR daemon 首次加载约 27s，常驻后识别延迟仅取决于录音时长。TTS daemon 同样常驻内存，内部分离合成线程与播放线程。

句子级流式 TTS。LLM 流式生成 token 时，系统将其按完整中文句子分组（25-90 字符/段），每攒齐一段即送入 TTS 合成。在 LLM 还在生成后续内容的同时，前面的段落已在合成或播放中，缩短了用户等待时间。

领域微调与安全约束。使用 5180 条开放许可的儿童故事样本对 Qwen2.5-1.5B-Instruct 做 LoRA 微调，训练过程中仅对 assistant 回答计算 loss。Modelfile 中设定较低的 temperature (0.18) 和较高的 repeat_penalty (1.16)，system prompt 要求故事内容安全、禁止输出表情符号与解释性文字。

D. 主要性能指标

表 I
K1 端侧主要性能指标

指标	数值
LLM 模型体积 (Q4_K_M)	986 MB
纯 LLM 首 token 延迟 (已预热)	2.88-2.98 s
纯 LLM 生成速度	3.34-3.38 tok/s
RAG 缓存命中后首 token 延迟	4.81 s
RAG+LLM+TTS 并发首 token	5.34-5.74 s
RAG+LLM+TTS 并发生成速度	1.64-1.67 tok/s
TTS 首段音频延迟	2.38-4.70 s
TTS 实时率 (RTF)	1.77-1.84
ASR 首次模型加载时间	约 27-28 s
训练数据 SFT 样本数	5180
RAG 知识卡片数	156

E. 主要创新点

1) 在 K1 RISC-V CPU 上实现了 ASR + RAG + 1.5B LLM + TTS 四模块端侧全链路部署，无需网络连接即可完成从语音输入到语音输出的完整交互。

- 2) 提出句子级流式 TTS 策略：LLM 逐 token 生成的同时，完整句子即时送入 TTS 后台合成与播放，以流水线方式覆盖各模块的串行延迟。
- 3) 设计了面向儿童动画 IP 的轻量 BM25 RAG 方案，不依赖 embedding 模型，直接将角色设定与安全约束注入 LLM 生成过程。
- 4) ASR 与 TTS 均采用 C 语言常驻 daemon 架构，消除了模型反复加载的冷启动开销，配合模型预热脚本将系统就绪时间控制在可接受范围内。

F. 设计流程

项目开发大致经历了四个阶段。先是数据准备与模型训练——收集 TinyStories Chinese 等开放数据集并清洗，在 RTX 4060 Ti 上对 Qwen2.5-1.5B-Instruct 做 LoRA 微调，合并权重后量化为 Q4_K_M GGUF。之后进入端侧模块开发，分别实现了 ASR C daemon、TTS C daemon、RAG 检索脚本和 LLM 调用脚本，并在 K1 板卡上逐模块调试。各模块调通后进行系统集成，编写按钮控制逻辑将各模块串联，开发 Qt6 界面，实现打断与重新开始机制。最后是性能优化与测试，主要工作包括模型预热、TTS 线程优先级调度、prefix 缓存等。

II. 系统组成及功能说明

A. 整体介绍

图 1 给出了系统的整体架构。系统在 K1 MUSE Pi Pro 板卡上运行 Bianbu Linux，核心数据通路为：实体按钮触发录音 → USB 麦克风采集 WAV → ASR daemon 识别文本 → BM25 RAG 检索知识卡片 → 本地 1.5B LLM 流式生成故事 → 句子分段 → TTS daemon 合成语音 → ffmpeg 转码 → aplay 通过 3.5 mm 耳机口播放。Qt6 前端读取后台 Python 进程的 stdout 事件，将技术状态翻译为儿童可理解的提示文字，并在 TTS 播放完一段音频后同步显示对应的故事文本。

各模块之间的协作关系如下：按钮控制脚本 (Python) 负责管理整个交互状态机，在按钮按下时启动录音，松开时停止录音并将 WAV 路径通过 stdin 送入 ASR daemon；ASR daemon 将识别结果以 JSON 格式通过 stdout 返回；按钮脚本解析识别文本后调用 RAG + LLM 脚本，后者以流式方式将生成的 token 通过 FIFO 送入 TTS daemon；TTS daemon 内部的合成线程与播放线程并行工作，将合成后的 PCM 音频经

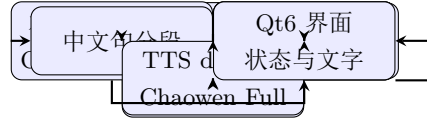


图 1. 系统整体架构图

ffmpeg 和 aplay 输出到硬件。Qt6 前端以独立进程运行，通过读取按钮脚本的 stdout 获取状态事件。

B. 硬件系统介绍

1) 硬件整体介绍：系统以进迭时空 K1 MUSE Pi Pro 为核心主控。K1 芯片集成 8 核 64 位 RISC-V CPU，提供 50K DMIPS 算力和 2TOPS AI 算力，配备 16 GB LPDDR4X 内存。板卡通过以下外设完成语音交互功能：

表 II
硬件外设配置

外设	接口	用途
USB 麦克风	USB-A	语音输入 (16 kHz)
3.5mm 耳机	音频口	语音输出播放
GPIO 35 按钮	40Pin	录音/识别/打断
NVMe SSD	M.2	存放模型文件

硬件接线比较简单：USB 麦克风插入板卡 USB-A 口，耳机或扬声器接 3.5 mm 音频口，按钮通过杜邦线连接 40Pin 排针的 GPIO 35 引脚，供电用 USB PD 12 V 3 A 电源适配器即可。

2) 电路模块介绍：本项目未涉及自制 PCB 电路设计，所有硬件均基于 K1 MUSE Pi Pro 板卡的原生接口。GPIO 按钮电路为最简单的上拉/下拉配置：按钮一端接 GPIO 35 引脚，另一端接 GND，软件中配置为 active low 模式，按下时 GPIO 电平拉低。音频输入输出分别由板载 USB Host 控制器和板载音频编解码器 (ES8326) 处理，无需额外的外围电路。

C. 软件系统介绍

1) 软件整体介绍：软件系统由五个主要模块组成，运行在 Bianbu Linux 上：

1) **ASR 模块**：C 语言编写的常驻 daemon (xiaodie_asr_daemon)，使用 sherpa-onnx C API 加载 Zipformer int8 模型，通过 stdin 接收 WAV 路径、stdout 输出 JSON 识别结果。

2) **LLM/RAG 模块**：Python 脚本 (xiaodie_rag_ollama_stream.py)，负责 BM25 检索 IP 知识卡片、构造 prompt、调用 Ollama 本地 HTTP API 进行流式推理，并将完整句子通过 FIFO 送入 TTS。

3) **TTS 模块**：C 语言编写的常驻 daemon (chaowen_tts_daemon)，使用 sherpa-onnx offline TTS C API 加载 Chaowen Full VITS 模型，内部包含合成线程和播放线程，通过 stdin 接收文本、输出合成状态日志。

4) **按钮控制模块**：Python 脚本 (xiaodie_button_story.py)，管理 GPIO 按钮状态机，协调录音、ASR、LLM 和 TTS 各环节的启动与打断。

5) **Qt6 界面**：C++ 编写的原生 GUI (xiaodie_qt_gui)，读取按钮脚本的 stdout 事件，翻译为“小蝶正在听你说话”“小蝶正在想故事”等儿童友好提示。

2) **ASR 模块**：ASR daemon 使用 sherpa-onnx 提供的 streaming Zipformer transducer 模型（中英文 + 标点，int8 量化）。模型文件包括 encoder.int8.onnx、decoder.onnx、joiner.int8.onnx 和 tokens.txt，provider 设为 CPU，解码方式为 greedy_search，线程数为 4。

daemon 启动后常驻内存。按钮脚本将录音 WAV 路径通过 stdin 发送给 daemon，daemon 读取 WAV 并执行识别，结果以 JSON 格式输出：

```
{"ok":true, "text":"佩奇和乔治整理玩具",
  "audio_s":3.2, "elapsed_s":1.1, "rtf":0.34}
```

ASR 模块的关键优化包括：（1）录音结束后修复 arecord 被 SIGINT 打断时的 WAV header 长度字段；（2）在录音尾部补充 500 ms 静音 padding，提升松开按钮后末尾文字的识别完整度；（3）识别失败时自动重启 daemon。

3) **LLM 与 RAG 模块**：LLM 推理通过 Ollama 框架完成。模型文件为 qwen2_5-1_5b-xiaodie-story-Q4_K_M.gguf（约 986 MB），在 Ollama 中注

册为 xiaodie-story-1.5b:latest。Modelfile 配置了如下推理参数：上下文长度 4096（端侧测试时限制到 1024 以降低延迟）、temperature 0.18、top_p 0.72、repeat_penalty 1.16。system prompt 要求输出安全的儿童故事正文，禁止输出 emoji 和解释性文字。

RAG 模块在调用 LLM 之前执行。输入为 ASR 识别文本，BM25 检索引擎对 156 张 IP 知识卡片进行匹配，默认取 top_k=2 张卡片（每张限 80 字符），按 franchise 过滤后强制加入 curated_fact、safety_policy 和 alias_policy 类型的卡片。检索结果注入 prompt，明确要求模型“只使用知识卡片中出现的人物、地点、关系和设定”。

BM25 的 token 化函数同时提取英文词、中文单字和中文 bigram，以提高中英文别名（如 Peppa Pig/小猪佩奇）的匹配率。该方案不依赖 embedding 模型，在 K1 端无额外的向量计算开销。

LLM 流式推理过程中，每个 token 经过 emoji/ANSI 控制字符/Markdown 符号过滤后，进入句子分组器。分组器按 2-4 个完整中文句子组成一个 chunk（25-90 字符），确保每个 TTS 输入都是语义完整的段落，不会在半句话中间切断。

4) TTS 模块：TTS daemon 使用 sherpa-onnx offline TTS C API 和 Chaowen Full 中文语音模型。程序启动时加载模型到内存，此后所有合成请求共享同一模型实例。

daemon 通过 stdin 接收文本和控制指令。普通 UTF-8 文本行追加到待合成缓冲区；::flush 强制合成剩余文本；::mark TOKEN 在播放队列中插入标记，用于通知 UI 某段音频已播完；::reset 清空待合成文本和任务队列。

daemon 内部包含两个工作线程：合成线程将文本转为 PCM 音频数据，播放线程将 PCM 经 ffmpeg 重采样后通过 aplay 送到硬件音频设备（ES8326 编解码器）。两个线程通过队列连接，合成与播放可并行进行。

TTS 模块与 Qt 界面之间通过::mark 协议实现同步：LLM 每生成完一段文本并送入 TTS 后，在队列末尾插入一个 mark；TTS 播放完该段音频后输出 mark_done 日志；Qt 前端据此显示对应段落的文字，并在所有段落播完后显示“故事讲完了”。

5) 按钮控制与状态机：按钮控制脚本管理整个交互的生命周期。状态机的工作流程如下：

1) 程序启动时，先启动 TTS 服务和 ASR daemon，等

待模型加载完成。

- 2) 儿童按住按钮：若当前有未完成任务，先打断录音、ASR、LLM 和 TTS；随后使用 arecord 开始录音。
- 3) 儿童松开按钮：发送 SIGINT 停止 arecord，修复 WAV header，将 WAV 路径送入 ASR daemon。
- 4) ASR 返回文本：若文本过短则通过 TTS 播放提示“没有听清楚，可以说一遍吗”；若文本有效则进入故事生成。
- 5) 再次按下按钮：强制终止当前故事生成进程，重启或清空 TTS 服务，重新开始录音。

打断机制的实现依赖 generation token：每次新的交互分配一个新 token，旧线程检测到 token 变化后停止输出，避免上一轮的残余文本或音频影响新的交互。

6) Qt6 原生界面：用户界面采用 Qt6 C++ 编写，避免了 Web 方案的浏览器开销。界面左侧显示进迭时空 logo、标题、“开始使用”和“结束使用”按钮；右侧显示当前状态和故事文本。

Qt 进程以子进程方式启动按钮控制脚本，读取其 stdout 中的事件标签。事件到界面提示的映射示例：recording...→“小蝶正在听你说话”；thinking story→“小蝶正在想一个适合你的故事”；[xiaodie_played] 文本 → 显示该段故事文字；[xiaodie_tts_done]→“故事讲完了，还可以继续点播新故事”。所有面向用户的文字均不包含 ASR、GPIO、Chaowen Full 等技术术语。

III. 完成情况及性能参数

A. 整体介绍

系统在 K1 MUSE Pi Pro 板卡上完成了端到端集成调试，各模块运行正常。硬件连接为 USB 麦克风、3.5mm 耳机、GPIO 按钮和 USB PD 供电；软件方面，Bianbu Linux 上部署了 ASR daemon、TTS daemon、Ollama 框架、RAG 检索脚本、按钮控制脚本和 Qt6 界面程序。

B. 工程成果

1) 软件成果：表 III 列出了各模块的核心文件及其在 K1 板卡上的部署路径。

C. 特性成果与性能测试

1) 模型训练结果：表 IV 汇总了 1.5B 模型的训练配置与结果。

表 III
软件模块部署清单

模块	核心文件	K1 部署路径
ASR	xiaodie_asr_daemon (C 二进制)	/home/vicky/xiaodie/asr/bin/
LLM/RAG	xiaodie_rag_ollama_stream.py	/home/vicky/xiaodie/llm/
TTS	chaowen_tts_daemon (C 二进制)	/home/vicky/xiaodie/tts/fast/bin/
按钮控制	xiaodie_button_story.py	/home/vicky/xiaodie/app/
Qt 界面	xiaodie_qt_gui (C++ 二进制)	/home/vicky/xiaodie/app/
模型	Q4_K_M GGUF (986 MB)	/home/vicky/xiaodie/models/
RAG 数据	ip_knowledge_cards.jsonl	/home/vicky/xiaodie/rag/

表 IV
1.5B 模型训练配置与结果

项目	数值
基座模型	Qwen2.5-1.5B-Instruct
训练方式	LoRA ($r=16, \alpha=32$)
目标模块	q/k/v/o_proj
训练数据	4973 train + 207 eval
训练步数	2000
最佳 eval loss	1.882
训练耗时	571 s
量化格式	Q4_K_M GGUF
量化后体积	986 MB

训练过程中 eval loss 随步数逐步降低，从第 200 步的 2.011 降至第 2000 步的 1.882，未出现过拟合迹象。训练数据来源于 TinyStories Chinese (5207 条)、Chinese Cosmopedia (240 条) 和 Storybooks Chinese (8 条)，均为开放许可数据集。数据清洗阶段过滤了 HTML 标签、非中文文本、教学类非故事文本以及包含暴力、成人、隐私等不安全内容的样本。

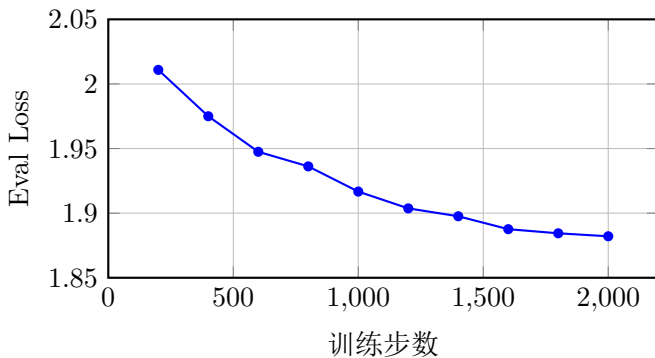


图 2. 1.5B 模型训练 eval loss 变化曲线

2) 端侧推理性能: 表 V 给出了不同运行配置下的 K1 端侧推理性能。

需要关注的几个现象: RAG 冷 prefix 场景下 TTFT 高达 50.52s, 原因是 Ollama 需要对整段 prompt 做首

次 KV cache 计算; 通过预热脚本预先执行一次带有相同 RAG prefix 的推理后, TTFT 可降至 4.81s。LLM 与 TTS 并发运行时, 由于 Chaowen Full 的合成过程也占用 K1 CPU 资源, LLM 生成速度从约 3.2 tokens/s 降至约 1.6 tokens/s。端侧部署时采用句子级流式 TTS 策略应对这一速度下降——LLM 生成首批完整句子后即开始 TTS 合成, 后续句子的生成与前序句子的合成可并行进行。

表 VI
CHAOWEN FULL TTS 端侧合成性能

输入文本	延迟 (s)	时长 (s)	RTF
佩奇正在整理玩具。	4.70	2.77	1.84
乔治也来帮忙。	2.38	1.35	1.77

3) TTS 性能: Chaowen Full 模型的音质经试听对比确认优于其他轻量 TTS 方案, 但 RTF 大于 1, 合成速度慢于实时播放。为此系统采用分段合成加后台排队播放的方式: 播放当前段时合成线程已在处理下一段文本, 减少用户实际感受到的等待。

4) 数据合规: 训练数据全部来源于明确许可的开放数据集, 未使用盗版小说站、商业童话站或版权不明的出版物。表 VII 列出了各数据源的许可协议。

表 VII
训练数据来源与许可

数据源	许可协议	样本
TinyStories CN	CDLA-Sharing-1.0	5207
CN Cosmopedia	Apache-2.0	240
Storybooks CN	MIT	8

清洗过程中过滤掉的不安全内容包括: 儿童危险行为 441 条、暴力内容 331 条、教学类非故事文本 194

表 V
K1 端侧推理性能对比

配置	TTFT (s)	生成速度 (tokens/s)	Prompt Eval (tokens/s)
纯 LLM 已预热（短故事）	2.98	3.34	87.69
纯 LLM 已预热（中等故事）	2.88	3.38	91.51
RAG 冷 prefix	50.52	3.56	14.01
RAG 缓存命中	4.81	3.20	158.53
RAG + LLM + TTS 并发	5.34–5.74	1.64–1.67	131–141

条、英文占比过高 76 条、医疗建议 36 条、成人内容 30 条、隐私相关 8 条、政治宣传 2 条和仇恨言论 1 条。

IV. 总结

A. 可扩展之处

系统目前还有几个方向可以进一步拓展。模型方面，项目同时训练并导出了 Qwen3-4B 的 GGUF 量化模型（Q4_K_M 约 2.5GB），在内存更充裕的板卡或下一代 K3 芯片上可直接切换到 4B 模型，故事生成质量会有明显提升。RAG 知识库目前包含 5 个动画 IP 的 156 张卡片，后续可扩展更多 IP 或引入科普、安全教育等非娱乐类知识主题。ASR 方面，当前使用的 Zipformer 模型对儿童语音的识别准确率仍有提升余地，可通过采集儿童语音数据进行针对性微调。TTS 方面，若后续出现适配 RISC-V 的轻量神经网络语音克隆方案，可实现更丰富的角色配音效果。交互层面，目前仅支持单轮对话，可扩展为多轮对话模式以实现更自然的故事续编能力。

B. 心得体会

本项目从选题到调试完成，先后经历了数据准备、模型训练、端侧移植和系统集成。以下记录开发过程中的一些具体体会。

在模型选型阶段，我们原计划使用 Qwen3-4B 作为端侧模型。4B 模型在台式机上生成的故事质量确实更好，句式更丰富，角色描写也更细致。但部署到 K1 板卡后，4B 模型的内存占用和推理延迟都给有限的硬件资源带来了压力，特别是与 TTS 并发运行时 CPU 竞争严重。最终我们决定以 1.5B 作为端侧主力模型，4B 则保留为质量对照。实际使用中发现，端侧场景下模型选型不能只看生成质量，还要考虑加载时间、推理速度以及和其他模块的 CPU 竞争。

数据清洗是另一个投入大量时间的环节。开放数据集集中混杂了相当比例的教学类文本、过短或过长的片段

以及包含不适宜儿童内容的样本。我们编写了多级过滤规则，包括中文字符比例检测、关键词安全过滤和中文 shingle 近重复去重。经过清洗后保留的 5455 条样本虽然在总量上不算大，但质量比较齐整，训练出的模型在儿童故事场景下表现尚可。

ASR 和 TTS 改成 daemon 常驻后，端侧体验改善了不少。改造前每次交互都要重新加载模型，ASR 加载超过 27 秒，TTS 也要数秒初始化，实际使用时等待感很强。改为常驻 daemon 后，模型只在应用启动时加载一次，后续录音识别和语音合成均在秒级完成。daemon 化在嵌入式开发中算不上新做法，但在端侧 AI 场景下确实必要。

句子级流式 TTS 是应对 K1 算力限制的主要手段。Chaowen Full 的 RTF 大于 1，若等 LLM 生成完整故事再统一合成，等待时间过长。把 LLM 输出按完整句子分段后即时送入 TTS 后台，播放前面段落的同时 LLM 继续生成后续内容，用户实际感受到的等待就短了很多。

Qt 界面开发时我们有意避开了所有技术术语。面向幼儿的产品不该出现“ASR”“TTS”“GPIO”之类的词，换成“小蝶正在听你说话”“小蝶正在想故事”这样的说法更合适。文字和语音的同步也花了不少功夫——TTS 播放完某段音频后界面才显示对应文字，否则文字先于声音出现会让人觉得不协调。

做完这个项目后，我们对 RISC-V 平台跑 AI 应用有了比较具体的感受。K1 本身不是为大模型推理设计的芯片，但通过模型量化、常驻进程、prefix 缓存和流式处理这几项工程优化，在端侧跑通包含 ASR、LLM 和 TTS 的完整语音链路是可行的。记录下这些踩坑经验，如果后续有人在 RISC-V 上做类似的端侧 AI 应用，或许能少走一些弯路。

参考文献

- [1] Qwen Team, “Qwen2.5 Technical Report,” arXiv:2412.15115, 2024.

- [2] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” arXiv:2106.09685, 2021.
- [3] T. Dettmers, A. Pagnoni, A. Holtzman, et al., “QLoRA: Efficient Finetuning of Quantized LLMs,” arXiv:2305.14314, 2023.
- [4] F. Kang, H. Yang, D. Groeneveld, et al., “llama.cpp: Efficient LLM Inference on Consumer Hardware,” GitHub, 2023.
- [5] Kaldi Team, “sherpa-onnx: Speech-to-Text, Text-to-Speech, and Keyword Spotting using Next-Gen Kaldi with ONNX,” GitHub, 2024.
- [6] S. E. Robertson and S. Walker, “Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval,” in *Proc. SIGIR*, 1994, pp. 232–241.
- [7] SpacemiT, “K1 Technical Reference Manual,” 进迭时空官方文档, 2024.
- [8] SpacemiT, “Bianbu Linux SDK Developer Guide,” 进迭时空官方文档, 2024.
- [9] M. Roubert, et al., “VITS: Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech,” in *Proc. ICML*, 2021.
- [10] R. Eloff, et al., “TinyStories: How Small Can Language Models Be and Still Speak Coherent English?” arXiv:2305.07759, 2023.
- [11] P. Lewis, E. Perez, A. Piktus, et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Proc. NeurIPS*, 2020.
- [12] Z. Zhan, et al., “Zipformer: A Faster and Better Encoder for Automatic Speech Recognition,” arXiv:2310.11230, 2023.
- [13] The Qt Company, “Qt 6 Documentation,” qt.io, 2024.
- [14] Ollama Team, “Ollama: Run Large Language Models Locally,” GitHub, 2024.